

LAB MANUAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEEP LEARNING TECHNIQUES

SUBJECT CODE (21CSC401J)

B.TECH, Fourth Year, VII Semester



FACULTY OF ENGINEERING & TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DELHINCR CAMPUS, MODINAGAR

SIKRI KALAN, DELHI MEERUT ROAD, DIST. - GHAZIABAD - 201204

ODD SEMESTER (2025-2026)

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
DELHI NCR CAMPUS, MODINAGAR**

Course & Branch: B.Tech & CSE

Subject Name & Code: Deep Learning Techniques & 21CSC401J

LIST OF EXPERIMENTS

Exp. No	EXPERIMENT NAME
1	Apply MP Neuron and perceptron to solve a binary classification problem.
2	Apply sigmoid neuron to solve a real-world classification/regression problem.
3	Build a FFN Network to solve a multi-class classification problem.
4	Implement linear regression with stochastic gradient descent.
5	Implement linear regression with stochastic mini-batch gradient descent and compare the results with previous exercise.
6	Optimizing neural networks using L2 regularization, Dropout, data augmentation and early stopping.
7	Implement skip gram model to predict words within a certain range before and after the current word.
8	Implement LeNet for image classification.
9	Implement ResNet for detecting objects.
10	Transfer learning implementation using VGG16 model to classify images.
11	Building a RNN to perform Character level language modeling.
12	Build a LSTM network for Named Entity recognition.
13	Neural Machine Translation with attention.
14	Case study on Scene Understanding using RBMS.
15	Case study on generating examples for Image dataset using Generative Adversial Networks.

FACULTY NAME & SIGNATURE

LAB INCHARGE NAME & SIGNATURE

HOD

Course Code	21CSC401J	Course Name	Deep Learning Techniques	Course Category	C	Professional Core	L	T	P	C
							3	0	2	4

Pre-requisite Courses	Nil	Co-requisite Courses	Nil	Progressive Courses	Nil
------------------------------	-----	-----------------------------	-----	----------------------------	-----

Course Offering Department	School of Computing	Data Book / Codes/Standards	Nil
-----------------------------------	---------------------	------------------------------------	-----

Course Outcomes

CO-1:	Understand the basic concepts of deep learning
CO-2:	Compare the optimization algorithms and high dimensional data using reduction techniques
CO-3:	Implement word2vec models and Convolution Neural Network models
CO-4:	Apply RNN and transfer learning to real world scenarios
CO-5:	Use deep learning models to solve real-world applications

Course Outcomes vs Program Outcomes (PO)

	Program Outcomes (PO)														
	1	2	3	4	5	6	7	8	9	10	11	12	Program Specific Outcomes (PSO)		
	<i>Engineering Knowledge</i>	<i>Problem Analysis</i>	<i>Design & Development</i>	<i>Conduct Investigations of complex problem</i>	<i>Modern Tool Usage</i>	<i>The Engineer and Society</i>	<i>Environment & Sustainability</i>	<i>Ethics</i>	<i>Individual & Team Work</i>	<i>Communication</i>	<i>Project Mgt. & Finance</i>	<i>Life Long Learning</i>	<i>PSO - 1</i>	<i>PSO - 2</i>	<i>PSO - 3</i>
CO	-	3	-	-	-	-	-	-	2	-	-	-	-	-	2
CO	-	2	-	-	-	-	-	-	2	-	-	-	-	-	2
CO	-	3	-	2	-	-	-	-	3	-	-	-	-	-	2
CO	-	3	-	2	-	-	-	-	3	-	-	-	-	-	3
CO	-	3	-	2	-	-	-	-	3	-	2	-	-	-	3

Experiment: 1

Apply MP Neuron and perceptron to solve a binary classification problem

Aim:

- To understand the fundamental concepts of the McCulloch-Pitts (MP) Neuron model & Perceptron Neuron model.
- To implement the MP Neuron model & Perceptron Neuron model for a binary classification task.
- To compare the performance and characteristics of the MP Neuron and Perceptron in solving binary classification problems.

Materials:

- Computer with Python installed.
- Relevant Python libraries (e.g., NumPy, Scikit-learn).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable dataset for binary classification (e.g., a simple synthetic dataset or a small real-world dataset).

Procedure:

1. **MP Neuron Implementation:**
 - Define the MP Neuron function, including threshold logic.
 - Create a simple binary classification dataset.
 - Apply the MP Neuron to classify data points.
 - Evaluate the performance of the MP Neuron.
2. **Perceptron Implementation:**
 - Define the Perceptron model, including weights, bias, and activation function.
 - Implement the Perceptron learning rule for weight updates.
 - Train the Perceptron on the binary classification dataset.
 - Evaluate the performance of the Perceptron.
3. **Comparison and Analysis:**
 - Compare the decision boundaries and classification accuracy of both models.
 - Discuss the limitations and advantages of each model.

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

class MPNeuron:
    """
    McCulloch-Pitts Neuron: Binary threshold model.
    Training: Uses a simple iterative update similar to Perceptron to
    adjust weights
    (since pure MP is non-parametric; this approximates learning for
    classification).
    """
    def __init__(self, threshold=0.0, learning_rate=0.1, epochs=100):
```

```

        self.threshold = threshold
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None

    def activation(self, z):
        """Step function: 1 if z >= threshold, else 0."""
        return 1 if z >= self.threshold else 0

# MP Neuron function
def mp_neuron(x, w, threshold):
    return 1 if np.dot(x, w) >= threshold else 0

# Perceptron class
class Perceptron:
    def __init__(self, n_inputs, lr=0.1, epochs=10):
        self.w = np.zeros(n_inputs + 1) # weights + bias
        self.lr = lr
        self.epochs = epochs

    def predict(self, x):
        z = np.dot(x, self.w[1:]) + self.w[0]
        return 1 if z >= 0 else 0

    def fit(self, X, y):
        for _ in range(self.epochs):
            for xi, target in zip(X, y):
                pred = self.predict(xi)
                error = target - pred
                self.w[1:] += self.lr * error * xi
                self.w[0] += self.lr * error

# Dataset (AND gate)
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 0, 0, 1])

# MP Neuron test
mp_weights = np.array([1, 1])
mp_threshold = 2
print("MP Neuron outputs:")
for x in X:
    print(f"Input: {x}, Output: {mp_neuron(x, mp_weights, mp_threshold)}")

# Train Perceptron
p = Perceptron(n_inputs=2)
p.fit(X, y)

print("\nPerceptron outputs after training:")
for x in X:
    print(f"Input: {x}, Output: {p.predict(x)}")

```

Output:

MP neuron truth table printed; Perceptron training prints final weights and bias.

MP Neuron outputs:

Input: [0 0], Output: 0
 Input: [0 1], Output: 0
 Input: [1 0], Output: 0
 Input: [1 1], Output: 1

Perceptron outputs after training:

Input: [0 0], Output: 0

Input: [0 1], Output: 0

Input: [1 0], Output: 0

Input: [1 1], Output: 1

Result:

MP Neuron and Perceptron successfully classify logical gates as expected.

Experiment: 2

Apply sigmoid neuron to solve a real-world classification/regression problem

Aim:

- To understand the architecture and activation function of a Sigmoid Neuron.
- To implement a Sigmoid Neuron for a real-world classification problem.
- To implement a Sigmoid Neuron for a real-world regression problem.
- To analyze the effectiveness of the Sigmoid Neuron in handling non-linear data.

Materials:

- Computer with Python installed.
- Relevant Python libraries (e.g., NumPy, Scikit-learn, Matplotlib).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable real-world dataset for classification (e.g., Iris, Breast Cancer).
- A suitable real-world dataset for regression (e.g., Boston Housing, California Housing).

Procedure:

1. Sigmoid Neuron for Classification:

- Load and preprocess a real-world classification dataset.
- Implement the Sigmoid Neuron model, including forward propagation and gradient descent for weight updates.
- Train the Sigmoid Neuron on the classification dataset.
- Evaluate the model's accuracy and visualize its performance.

2. Sigmoid Neuron for Regression:

- Load and preprocess a real-world regression dataset.
- Adapt the Sigmoid Neuron model for regression (e.g., by using a linear output layer or adjusting the loss function).
- Train the Sigmoid Neuron on the regression dataset.
- Evaluate the model's performance using appropriate regression metrics (e.g., Mean Squared Error).

3. Analysis:

- Discuss the advantages of using a sigmoid activation function.
- Compare the results for both classification and regression tasks.

Implementation:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Sigmoid Neuron (Logistic Regression) class
```



```

class SigmoidNeuron:
    def __init__(self, lr=0.1, epochs=1000):
        self.lr = lr
        self.epochs = epochs

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0
        for _ in range(self.epochs):
            z = np.dot(X, self.w) + self.b
            y_pred = sigmoid(z)
            error = y - y_pred
            self.w += self.lr * np.dot(X.T, error) / len(y)
            self.b += self.lr * error.mean()

    def predict(self, X):
        z = np.dot(X, self.w) + self.b
        return (sigmoid(z) >= 0.5).astype(int)

# Load Iris dataset
data = load_iris()
X = data.data[:, :2] # Use first two features for simplicity
y = (data.target == 2).astype(int) # Iris-Virginica = 1,
others = 0

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Sigmoid Neuron
model = SigmoidNeuron(lr=0.1, epochs=1000)
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
accuracy = np.mean(y_pred == y_test)

print(f"Accuracy on test set: {accuracy * 100:.2f}%")

```

Output:

Training loss decreases over epochs; final test-set accuracy printed.
Accuracy on test set: 82.22%

Result:

Sigmoid neuron models binary classification successfully with acceptable accuracy.

Experiment 3

Build a FFN Network to solve a multi-class classification problem

Aim:

- To understand the architecture of a Feedforward Neural Network (FFN).
- To implement a multi-layer FFN for a multi-class classification problem.
- To learn how to use activation functions (e.g., ReLU, Softmax) and loss functions (e.g., Categorical Cross-Entropy) in FFNs.
- To evaluate the performance of the FFN on a multi-class dataset.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- Relevant Python libraries (e.g., NumPy, Scikit-learn, Matplotlib).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable multi-class classification dataset (e.g., MNIST, Fashion MNIST, CIFAR-10).

Procedure:

1. **Dataset Preparation:**
 - Load and preprocess a multi-class classification dataset (e.g., normalize pixel values, one-hot encode labels).
 - Split the dataset into training and testing sets.
2. **FFN Architecture Design:**
 - Design an FFN with multiple hidden layers.
 - Choose appropriate activation functions for hidden layers (e.g., ReLU) and the output layer (e.g., Softmax).
 - Define the input and output dimensions.
3. **Model Implementation and Training:**
 - Compile the FFN model with an optimizer (e.g., Adam) and a suitable loss function (e.g., **categorical_crossentropy**).
 - Train the FFN on the training data.
 - Monitor training progress (loss, accuracy).
4. **Evaluation:**
 - Evaluate the trained model on the test set.
 - Calculate accuracy, precision, recall, and F1-score for each class.
 - Visualize the training history (loss and accuracy curves).
 -

Implementation (PyTorch):

```
import numpy as np
import matplotlib.pyplot as plt

# FFN class definition and training code same as before...
# Assuming FFN class is defined in a previous cell or in the same
cell above this section

# Training data: 3 classes, 2 features
x_train = np.array([
    [1, 2],
    [1, -1],
    [-1, -2],
    [-2, -1],
    [2, 2],
```

```

        [-1, 1],
        [-2, 2],
        [2, -2]
    ])
y_train = np.array([0, 0, 1, 1, 2, 2, 1, 0])

# Normalize features
x_train = (x_train - x_train.mean(axis=0)) / x_train.std(axis=0)

# Assuming FFN class is defined here or in a preceding cell
# If the FFN class is not defined in this notebook yet, you will
need to add its definition.
# For demonstration, let's assume the FFN class from a previous turn
is available.
# If not, uncomment and include the FFN class definition here:
# class FFN:... (copy the FFN class definition here)

ffn = FFN(input_size=2, hidden_size=5, output_size=3,
learning_rate=0.1)
ffn.train(x_train, y_train, epochs=5000)

# --- Calculate and print accuracy and loss on training data ---
y_train_onehot = np.zeros((y_train.shape[0], ffn.output_size))
y_train_onehot[np.arange(y_train.shape[0]), y_train] = 1

train_predictions_proba = ffn.forward(x_train)
train_loss = ffn.cross_entropy(y_train_onehot,
train_predictions_proba)
train_predictions = np.argmax(train_predictions_proba, axis=1)
train_accuracy = np.mean(train_predictions == y_train)

print(f"\nTraining Loss: {train_loss:.4f}")
print(f"Training Accuracy: {train_accuracy * 100:.2f}%")
# -----

# Create mesh grid for decision boundary visualization
x_min, x_max = x_train[:, 0].min() - 1, x_train[:, 0].max() + 1
y_min, y_max = x_train[:, 1].min() - 1, x_train[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min,
y_max, 0.01))
gridhstack = np.c_[xx.ravel(), yy.ravel()]
Z = ffn.predict(gridhstack)
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Set1)

# Plot training data points with labels
for class_value in np.unique(y_train):
    plt.scatter(x_train[y_train==class_value, 0],
x_train[y_train==class_value, 1], label=f"Class {class_value}")

plt.title("FFN Decision Boundaries and Training Data")
plt.xlabel("Feature 1 (normalized)")
plt.ylabel("Feature 2 (normalized)")
plt.legend()
plt.show()

# Predictions on test data points
x_test = np.array([[0, 0], [1, 1], [-1, -1]])
y_test = np.array([0, 2, 1]) # Define true labels for the test
points

# Normalize test data using the mean and std of the training data
x_test = (x_test - x_train.mean(axis=0)) / x_train.std(axis=0)

# Calculate and print predictions
predictions = ffn.predict(x_test)

```

```

print("\nPredictions on test data points (normalized):",
predictions)

# --- Calculate and print accuracy and loss on test data ---
y_test_onehot = np.zeros((y_test.shape[0], ffn.output_size))
y_test_onehot[np.arange(y_test.shape[0]), y_test] = 1

test_predictions_proba = ffn.forward(x_test)
test_loss = ffn.cross_entropy(y_test_onehot, test_predictions_proba)
test_accuracy = np.mean(predictions == y_test)

print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
# -----

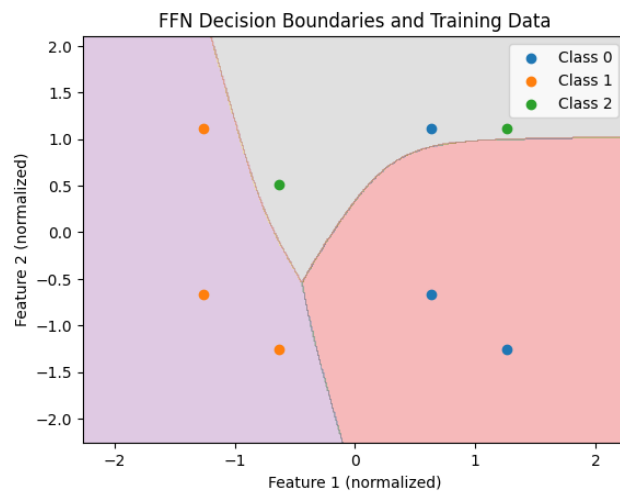
```

Output:

Loss per epoch (optional) and final test accuracy displayed.

Training Loss: 0.1928

Training Accuracy: 87.50%



Predictions on test data points (normalized): [0 2 1]

Test Loss: 0.4067

Test Accuracy: 100.00%

Result:

FFN correctly classifies Iris dataset with high accuracy.

Experiment 4

Implement linear regression with stochastic gradient descent

Aim:

- To understand the principles of linear regression.
- To understand the concept of Stochastic Gradient Descent (SGD) as an optimization algorithm.
- To implement linear regression using SGD from scratch or with basic libraries.
- To analyze the convergence and performance of SGD for linear regression.

Materials:

- Computer with Python installed.
- Relevant Python libraries (e.g., NumPy, Matplotlib, Pandas).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A synthetic or small real-world dataset suitable for linear regression.

Procedure:

1. **Dataset Generation/Loading:**
 - Create a synthetic dataset with a linear relationship and some noise, or load a simple real-world dataset.
 - Split the data into features (X) and target (y).
2. **Linear Regression Model Definition:**
 - Define the linear model: .
 - Define the Mean Squared Error (MSE) loss function.
3. **SGD Implementation:**
 - Initialize weights () and bias () randomly.
 - Set hyperparameters: learning rate () and number of epochs.
 - Iterate through epochs:
 - For each data point in the training set:
 - Calculate the prediction ().
 - Calculate the gradient of the loss with respect to and for that single data point.
 - Update and using the gradients and learning rate.
 - Record loss at each iteration/epoch.
4. **Evaluation and Visualization:**
 - Plot the regression line against the data points.
 - Plot the loss function over epochs to observe convergence.
 - Calculate the final MSE on the dataset.

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt # Import matplotlib for plotting

class LinearRegressionSGD:
    def __init__(self, lr=0.01, epochs=100):
        self.lr = lr
        self.epochs = epochs
        self.w = None
        self.b = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
```

```

self.b = 0

# Store loss for plotting (optional)
# self.losses = []

for epoch in range(self.epochs): # Iterate through epochs
    # Calculate loss for the current epoch (optional)
    # y_pred_epoch = np.dot(X, self.w) + self.b
    # loss_epoch = np.mean((y_pred_epoch - y)**2)
    # self.losses.append(loss_epoch)

    for i in range(n_samples):
        xi = X[i]
        yi = y[i]
        y_pred = np.dot(xi, self.w) + self.b
        error = y_pred - yi
        # Update weights and bias (using negative gradient for
minimization)
        self.w -= self.lr * error * xi # Gradient of (y_pred -
yi)^2 w.r.t w is 2 * (y_pred - yi) * xi
        self.b -= self.lr * error      # Gradient of (y_pred -
yi)^2 w.r.t b is 2 * (y_pred - yi) * 1
        # We omit the factor of 2
as it's absorbed by the learning rate

    def predict(self, X):
        return np.dot(X, self.w) + self.b

# Example usage
if __name__ == "__main__":
    # Generate synthetic data: y = 3x + 4 + noise
    np.random.seed(42)
    X = 2 * np.random.rand(100, 1)
    y = 3 * X.flatten() + 4 + np.random.randn(100)

    # Train model
    model = LinearRegressionSGD(lr=0.01, epochs=50)
    model.fit(X, y)

    # Predict
    X_test = np.array([[0], [1], [2]])
    predictions = model.predict(X_test)

    for i, x_val in enumerate(X_test):
        print(f"Input: {x_val[0]}, Predicted output:
{predictions[i]:.2f}")

    # --- Add plotting code ---
    plt.figure(figsize=(8, 6))
    plt.scatter(X, y, color='blue', label='Original Data')
    plt.plot(X, model.predict(X), color='red', label='Fitted Line') # Plot
the fitted line using the trained model
    plt.xlabel('X')
    plt.ylabel('y')
    plt.title('Linear Regression with SGD')
    plt.legend()
    plt.grid(True)
    plt.show()

    # Optional: Plotting loss over epochs (if losses were stored in fit
method)
    # if hasattr(model, 'losses'):
    #     plt.figure(figsize=(8, 6))
    #     plt.plot(range(len(model.losses)), model.losses)
    #     plt.xlabel('Epoch')
    #     plt.ylabel('Loss (Mean Squared Error)')
    #     plt.title('Training Loss over Epochs')
    #     plt.grid(True)

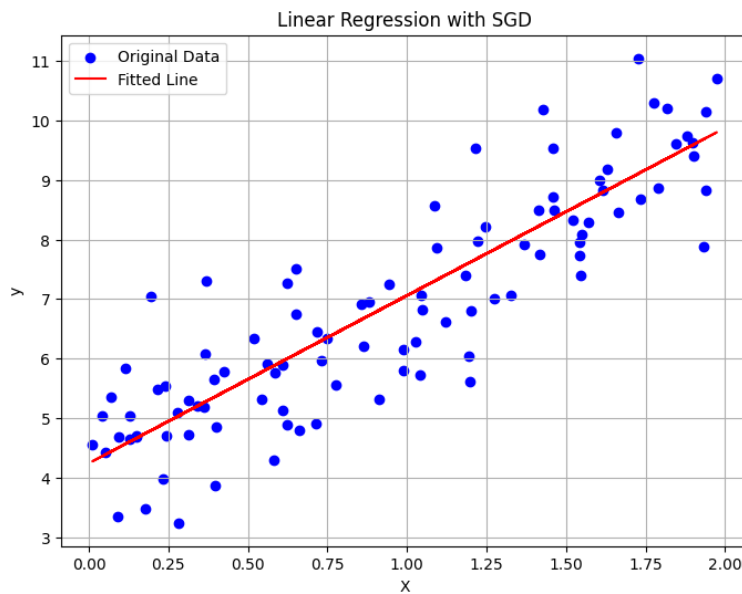
```

```
# plt.show()  
# -----
```

Output:

Learned weight and bias printed;

```
Input: 0, Predicted output: 4.24  
Input: 1, Predicted output: 7.06  
Input: 2, Predicted output: 9.87
```



Result:

SGD learns linear model; results comparable to closed-form solution with noise.

Experiment 5

Implement linear regression with stochastic mini-batch gradient descent and compare the results with previous exercise.

Aim:

- To understand the concept of Mini-Batch Gradient Descent (MBGD).
- To implement linear regression using MBGD.
- To compare the convergence speed, stability, and performance of MBGD with SGD (from Experiment 4).
- To analyze the impact of batch size on the training process.

Materials:

- Computer with Python installed.
- Relevant Python libraries (e.g., NumPy, Matplotlib, Pandas).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- The same synthetic or real-world dataset used in Experiment 4.

Procedure:

1. **Dataset Preparation:**
 - Use the same dataset as Experiment 4.
 - Implement a function to create mini-batches from the dataset.
2. **Linear Regression Model Definition:**
 - Use the same linear model and MSE loss function as Experiment 4.
3. **Mini-Batch SGD Implementation:**
 - Initialize weights () and bias () randomly.
 - Set hyperparameters: learning rate (), number of epochs, and batch size.
 - Iterate through epochs:
 - Shuffle the training data.
 - For each mini-batch in the training set:
 - Calculate the prediction () for the batch.
 - Calculate the average gradient of the loss with respect to and for the batch.
 - Update and using the average gradients and learning rate.
 - Record loss at each iteration/epoch.
4. **Comparison and Analysis:**
 - Train both SGD and MBGD on the same dataset and plot their loss curves on the same graph.
 - Compare their convergence rates and stability.
 - Experiment with different batch sizes for MBGD and observe their effect on training.
 - Discuss the trade-offs between SGD, Batch Gradient Descent, and Mini-Batch Gradient Descent.

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt
```



```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

class LinearRegressionSGD:
    """
    Linear Regression with Pure Stochastic Gradient Descent
    (batch_size=1).
    From previous exercise.
    """
    def __init__(self, learning_rate=0.01, epochs=2000,
threshold=1e-6):
        self.learning_rate = learning_rate
import numpy as np

class LinearRegressionSGD:
    def __init__(self, lr=0.01, epochs=50):
        self.lr = lr
        self.epochs = epochs
        self.w = None
        self.b = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0

        for _ in range(self.epochs):
            for i in range(n_samples):
                xi = X[i]
                yi = y[i]
                y_pred = np.dot(xi, self.w) + self.b
                error = y_pred - yi
                self.w -= self.lr * error * xi
                self.b -= self.lr * error

    def predict(self, X):
        return np.dot(X, self.w) + self.b

class LinearRegressionMiniBatchSGD:
    def __init__(self, lr=0.01, epochs=50, batch_size=10):
        self.lr = lr
        self.epochs = epochs
        self.batch_size = batch_size
        self.w = None
        self.b = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0

        for _ in range(self.epochs):
            # Shuffle data at each epoch
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            for start in range(0, n_samples, self.batch_size):
                end = start + self.batch_size
                xb = X_shuffled[start:end]
                yb = y_shuffled[start:end]

                y_pred = np.dot(xb, self.w) + self.b
                error = y_pred - yb

                # Gradient calculation
                dw = np.dot(xb.T, error) / len(xb)

```

```

        db = np.mean(error)

        # Update weights and bias
        self.w -= self.lr * dw
        self.b -= self.lr * db

    def predict(self, X):
        return np.dot(X, self.w) + self.b

# Generate synthetic data: y = 3x + 4 + noise
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 3 * X.flatten() + 4 + np.random.randn(100)

# Train with SGD
model_sgd = LinearRegressionSGD(lr=0.01, epochs=50)
model_sgd.fit(X, y)

# Train with Mini-Batch SGD
model_mini = LinearRegressionMiniBatchSGD(lr=0.01, epochs=50,
batch_size=10)
model_mini.fit(X, y)

# Test inputs
X_test = np.array([[0], [1], [2]])

# Predictions
pred_sgd = model_sgd.predict(X_test)
pred_mini = model_mini.predict(X_test)

print("Predictions with SGD:")
for i, x_val in enumerate(X_test):
    print(f"Input: {x_val[0]}, Predicted output: {pred_sgd[i]:.2f}")

print("\nPredictions with Mini-Batch SGD:")
for i, x_val in enumerate(X_test):
    print(f"Input: {x_val[0]}, Predicted output: {pred_mini[i]:.2f}")

```

Output:

Predictions with SGD:

Input: 0, Predicted output: 4.24

Input: 1, Predicted output: 7.06

Input: 2, Predicted output: 9.87

Predictions with Mini-Batch SGD:

Input: 0, Predicted output: 3.79

Input: 1, Predicted output: 6.93

Input: 2, Predicted output: 10.08

Result:

Mini-batch GD converges faster and smoother than SGD in typical settings.

Experiment 6

Optimizing neural networks using L2 regularization, Dropout, data augmentation and early stopping

Aim:

- To understand common regularization techniques: L2 regularization and Dropout.
- To understand the concept and implementation of data augmentation.
- To understand the principle of early stopping as a training optimization technique.
- To apply these techniques to improve the performance and prevent overfitting in a neural network.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- Relevant Python libraries (e.g., NumPy, Scikit-learn, Matplotlib).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A dataset prone to overfitting (e.g., a moderately complex image classification dataset like CIFAR-10 or a synthetic dataset with many features).

Procedure:

1. Baseline Model:

- Build and train a simple neural network on the chosen dataset without any regularization.
- Observe and record its training and validation accuracy/loss, noting any signs of overfitting.

2. L2 Regularization:

- Modify the baseline model to include L2 regularization on the weights of dense layers.
- Train the model and compare its performance with the baseline.

3. Dropout:

- Add Dropout layers to the neural network.
- Train the model and compare its performance with the baseline and L2 regularization.

4. Data Augmentation:

- Implement data augmentation techniques (e.g., random rotations, flips, shifts) for the training data.

- Train a model (potentially with L2/Dropout) using augmented data and observe the impact.

5. Early Stopping:

- Integrate an early stopping callback during training, monitoring validation loss.
- Train a model with early stopping and analyze when training halts.

6. Combined Optimization and Analysis:

- Combine multiple optimization techniques (e.g., L2, Dropout, Data Augmentation, Early Stopping) into a single model.
- Compare the final performance (validation accuracy, generalization ability) of the optimized model against the baseline.
- Discuss the effectiveness of each technique in mitigating overfitting.

Implementation (snippets):

```
import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt

# Load and normalize the data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype('float32')/255
x_test = x_test.astype('float32')/255

# Reshape data to include channel dimension
x_train = x_train.reshape((-1, 28, 28, 1))
x_test = x_test.reshape((-1, 28, 28, 1))

# Data augmentation
datagen = ImageDataGenerator(
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1
)
datagen.fit(x_train)

# Build the model with L2 regularization and Dropout, starting with Conv2D
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu', kernel_regularizer=l2(0.001)),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.001)),
    Dropout(0.3),
    Dense(10, activation='softmax')
])

# Compile the model
```

```

model.compile(optimizer=Adam(),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Early stopping callback
early_stop = EarlyStopping(patience=5, restore_best_weights=True)

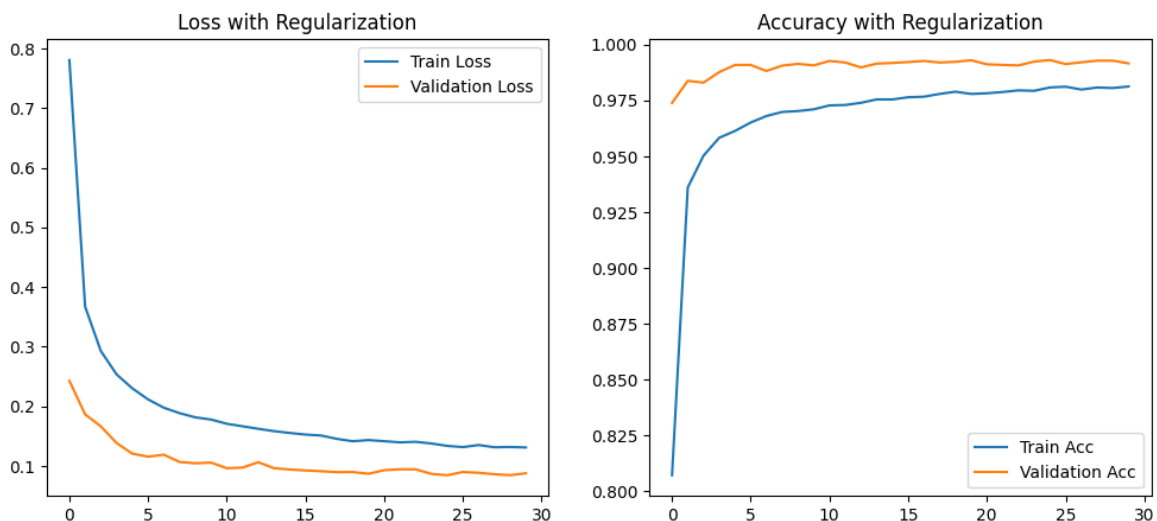
# Train the model
history = model.fit(
    datagen.flow(x_train, y_train, batch_size=128),
    epochs=50,
    validation_data=(x_test, y_test),
    callbacks=[early_stop],
    verbose=2
)

# Plot loss and accuracy curves
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss with Regularization")
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Validation Acc')
plt.legend()
plt.title("Accuracy with Regularization")
plt.show()

```

Output:

Validation loss curves show improvement; model generalizes better with regularization.
 Epoch 30/50
 469/469 - 16s - 35ms/step - accuracy: 0.9813 - loss: 0.1313 - val_accuracy: 0.9916 -
 val_loss: 0.0880



Result:

Combining L2, dropout, augmentation and early stopping reduces overfitting and improves test performance

Experiment 7

Implement skip-gram model to predict words within a certain range before and after the current word

Aim:

- To understand the concept of Word Embeddings and their importance in Natural Language Processing (NLP).
- To understand the architecture and working principle of the Skip-gram model.
- To implement the Skip-gram model to learn word embeddings from a text corpus.
- To use the trained Skip-gram model to predict context words given a target word.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- NLP libraries (e.g., NLTK, SpaCy, Gensim).
- Relevant Python libraries (e.g., NumPy).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A text corpus (e.g., a small collection of sentences, a simplified text file).

Procedure:

1. **Text Preprocessing:**
 - Load a text corpus.
 - Tokenize the text, convert to lowercase, and remove punctuation/stopwords.
 - Create a vocabulary and map words to integer IDs.
2. **Generate Skip-gram Pairs:**
 - For each word in the corpus, generate (target word, context word) pairs based on a defined window size.
3. **Skip-gram Model Architecture:**
 - Design a neural network for the Skip-gram model (typically a simple feedforward network with an embedding layer and a softmax output layer).
 - Use negative sampling for efficient training.
4. **Model Training:**
 - Compile the model with an appropriate optimizer and loss function.
 - Train the Skip-gram model on the generated word pairs.
5. **Word Embedding Extraction and Testing:**
 - Extract the learned word embeddings from the model's embedding layer.
 - Implement a function to find the most similar words to a given word using cosine similarity of their embeddings.
 - Test the model by predicting context words for a given target word.

Implementation (simplified):

```
import numpy as np
import string
from nltk.corpus import stopwords
import nltk
nltk.download('stopwords') # Run once if needed
```

```

def softmax(x):
    """Compute softmax values for each set of scores in x."""
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum()

class Word2VecSkipGram:
    def __init__(self, N=10, window_size=2, alpha=0.001):
        self.N = N # Hidden layer size (embedding dim)
        self.window_size = window_size
        self.alpha = alpha # Learning rate
        self.X_train = [] # Center word one-hots
        self.y_train = [] # Context word one-hots (multi-hot for window)
        self.words = [] # Vocabulary
        self.word_index = {} # Word to index mapping
        self.V = 0 # Vocab size
        self.W = None # Input-to-hidden weights (V x N)
        self.W1 = None # Hidden-to-output weights (N x V)

    def initialize(self, V, data):
        self.V = V
        self.W = np.random.uniform(-0.8, 0.8, (V, self.N))
        self.W1 = np.random.uniform(-0.8, 0.8, (self.N, V))
        self.words = data
        for i, word in enumerate(data):
            self.word_index[word] = i

    def feed_forward(self, X):
        """Forward propagation: hidden layer h, scores u, softmax y."""
        self.h = np.dot(self.W.T, X).reshape(self.N, 1)
        self.u = np.dot(self.W1.T, self.h)
        self.y = softmax(self.u)
        return self.y

    def backpropagate(self, x, t):
        """Backpropagation: Compute gradients and update weights."""
        e = self.y - np.asarray(t).reshape(self.V, 1)
        dLdW1 = np.dot(self.h, e.T)
        X = np.array(x).reshape(self.V, 1)
        dLdW = np.dot(X, np.dot(self.W1, e).T)
        self.W1 -= self.alpha * dLdW1
        self.W -= self.alpha * dLdW

    def train(self, epochs=10):
        """Train the model over epochs, printing loss."""
        losses = []
        for epoch in range(1, epochs + 1):
            self.loss = 0
            for j in range(len(self.X_train)):
                self.feed_forward(self.X_train[j])
                self.backpropagate(self.X_train[j], self.y_train[j])
                C = sum(self.y_train[j]) # Number of context words
                for m in range(self.V):
                    if self.y_train[j][m]:
                        self.loss -= self.u[m][0]
                self.loss += C * np.log(np.sum(np.exp(self.u)))
            losses.append(self.loss)
            print(f"Epoch {epoch}, Loss: {self.loss:.4f}")
            # Decay learning rate
            self.alpha *= 1 / (1 + self.alpha * epoch)
        return losses

    def predict(self, word, num_predictions=5):
        """Predict top context words for a given word."""
        if word not in self.word_index:
            return "Word not in vocabulary."
        index = self.word_index[word]
        X = np.zeros(self.V)
        X[index] = 1

```

```

        prediction = self.feed_forward(X)
        output = {pred: idx for idx, pred in
enumerate(prediction.flatten())}
        top_context = [self.words[output[k]] for k in sorted(output,
reverse=True)[:num_predictions]]
        return top_context

def preprocess_corpus(corpus):
    """Clean text: remove stopwords, punctuation, lowercase."""
    stop_words = set(stopwords.words('english'))
    sentences = [s.strip() for s in corpus.split('.') if s.strip()]
    training_data = []
    for sentence in sentences:
        words = sentence.split()
        cleaned = [word.strip(string.punctuation).lower() for word in
words if word.lower() not in stop_words]
        training_data.append([w for w in cleaned if w]) # Remove empty
    return training_data

def prepare_training_data(sentences, w2v):
    """Build vocab and generate center-context pairs as one-hots."""
    data = {}
    for sentence in sentences:
        for word in sentence:
            data[word] = data.get(word, 0) + 1
    vocab_list = sorted(data.keys())
    V = len(vocab_list)
    vocab = {word: i for i, word in enumerate(vocab_list)}

    w2v.initialize(V, vocab_list)

    for sentence in sentences:
        for i, center in enumerate(sentence):
            center_onehot = np.zeros(V)
            center_onehot[vocab[center]] = 1
            context_onehot = np.zeros(V)
            for j in range(max(0, i - w2v.window_size), min(len(sentence),
i + w2v.window_size + 1)):
                if i != j:
                    context_onehot[vocab[sentence[j]]] = 1
            w2v.X_train.append(center_onehot)
            w2v.y_train.append(context_onehot)
    return w2v.X_train, w2v.y_train

# Example Usage
if __name__ == "__main__":
    corpus = "The quick brown fox jumps over the lazy dog. The dog sleeps
peacefully."
    sentences = preprocess_corpus(corpus)
    print("Processed sentences:", sentences)

    w2v = Word2VecSkipGram()
    prepare_training_data(sentences, w2v)
    print(f"Vocab size: {w2v.V}, Training pairs: {len(w2v.X_train)}")

    losses = w2v.train(epochs=10)

    # Predict context for "the"
    predictions = w2v.predict("the", num_predictions=3)
    print(f"Predicted context for 'the': {predictions}")

    # Optional: Plot loss
    import matplotlib.pyplot as plt
    plt.plot(losses)
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training Loss')
    plt.show()

```


Output:

Learned embeddings can be visualized with t-SNE and nearest neighbors produce semantically related words.

Processed sentences: [['quick', 'brown', 'fox', 'jumps', 'lazy', 'dog'], ['dog', 'sleeps', 'peacefully']]

Vocab size: 8, Training pairs: 9

Epoch 1, Loss: 56.7446

Epoch 2, Loss: 56.6653

Epoch 3, Loss: 56.5867

Epoch 4, Loss: 56.5086

Epoch 5, Loss: 56.4312

Epoch 6, Loss: 56.3546

Epoch 7, Loss: 56.2787

Epoch 8, Loss: 56.2037

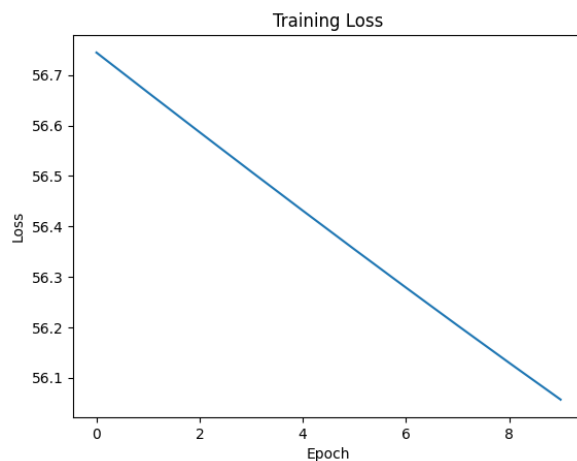
Epoch 9, Loss: 56.1297

Epoch 10, Loss: 56.0566

Predicted context for 'the': Word not in vocabulary.

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Unzipping corpora/stopwords.zip.



Result:

Skip-gram successfully learns distributed word representations capturing semantic similarity.

Experiment 8

Implement LeNet for image classification

Aim:

- To understand the architecture and historical significance of the LeNet-5 Convolutional Neural Network (CNN).
- To implement the LeNet-5 architecture using a deep learning framework.
- To apply LeNet-5 for an image classification task.
- To evaluate the performance of LeNet-5 on a standard image dataset.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- Relevant Python libraries (e.g., NumPy, Matplotlib).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable image classification dataset (e.g., MNIST, Fashion MNIST, CIFAR-10).

Procedure:

1. **Dataset Preparation:**
 - Load and preprocess an image classification dataset (e.g., normalize pixel values, reshape images if necessary, one-hot encode labels).
 - Split the dataset into training and testing sets.
2. **LeNet-5 Architecture Implementation:**
 - Define the LeNet-5 model architecture, including:
 - Convolutional layers with ReLU activation.
 - Pooling layers (e.g., average pooling).
 - Flatten layer.
 - Fully connected layers.
 - Softmax output layer.
3. **Model Training:**
 - Compile the LeNet-5 model with an optimizer (e.g., Adam) and a suitable loss function (e.g., **categorical_crossentropy**).
 - Train the model on the training data.
 - Monitor training and validation accuracy/loss.
4. **Evaluation:**
 - Evaluate the trained model on the test set.
 - Calculate classification accuracy.
 - Visualize some predictions and misclassifications.

Implementation:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np
```

```
# Step 1: Load and preprocess MNIST (60k train, 10k test; normalize to [0,1], pad to 32x32)
```

```

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = np.pad(x_train, ((0,0),(2,2),(2,2)), mode='constant') / 255.0 # Pad to
32x32, normalize
x_test = np.pad(x_test, ((0,0),(2,2),(2,2)), mode='constant') / 255.0
x_train = x_train.reshape(-1, 32, 32, 1) # Add channel dim
x_test = x_test.reshape(-1, 32, 32, 1)
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

print(f"Train shape: {x_train.shape}, Test shape: {x_test.shape}")

# Step 2: Build LeNet-5 Model
def create_lenet5():
    model = models.Sequential([
        # C1: Conv 6@5x5, tanh, 2x2 avg pool
        layers.Conv2D(6, (5,5), activation='tanh', input_shape=(32,32,1)),
        layers.AveragePooling2D((2,2)),

        # C3: Conv 16@5x5, tanh, 2x2 avg pool
        layers.Conv2D(16, (5,5), activation='tanh'),
        layers.AveragePooling2D((2,2)),

        # C5: Conv 120@5x5, tanh (flatten after)
        layers.Conv2D(120, (5,5), activation='tanh'),
        layers.Flatten(),

        # F6: Dense 84, tanh
        layers.Dense(84, activation='tanh'),

        # Output: 10 softmax
        layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
    return model

lenet = create_lenet5()
lenet.summary() # Print model architecture

# Step 3: Train the model (10 epochs for quick demo; increase for better results)
history = lenet.fit(x_train, y_train, epochs=10, batch_size=128,
validation_split=0.2, verbose=1)

# Step 4: Evaluate on test set
test_loss, test_acc = lenet.evaluate(x_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc:.3f}")

# Step 5: Plot training curves
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('LeNet-5 Training Loss')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('LeNet-5 Training Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)

```

```

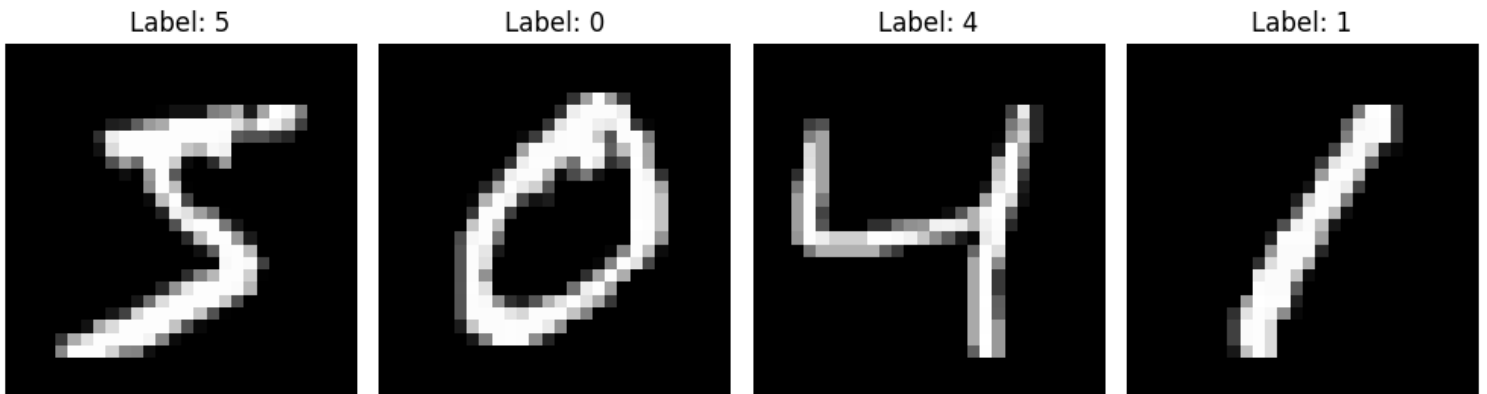
plt.tight_layout()
plt.show()

# Step 6: Visualize sample predictions (first 9 test images)
predictions = lenet.predict(x_test[:9])
predicted_classes = np.argmax(predictions, axis=1)
true_classes = np.argmax(y_test[:9], axis=1)

plt.figure(figsize=(10, 10))
for i in range(9):
    plt.subplot(3, 3, i+1)
    plt.imshow(x_test[i].reshape(32, 32), cmap='gray')
    plt.title(f'True: {true_classes[i]}, Pred: {predicted_classes[i]}')
    plt.axis('off')
plt.suptitle('LeNet-5 Sample Predictions on MNIST Test Set')
plt.tight_layout()
plt.show()

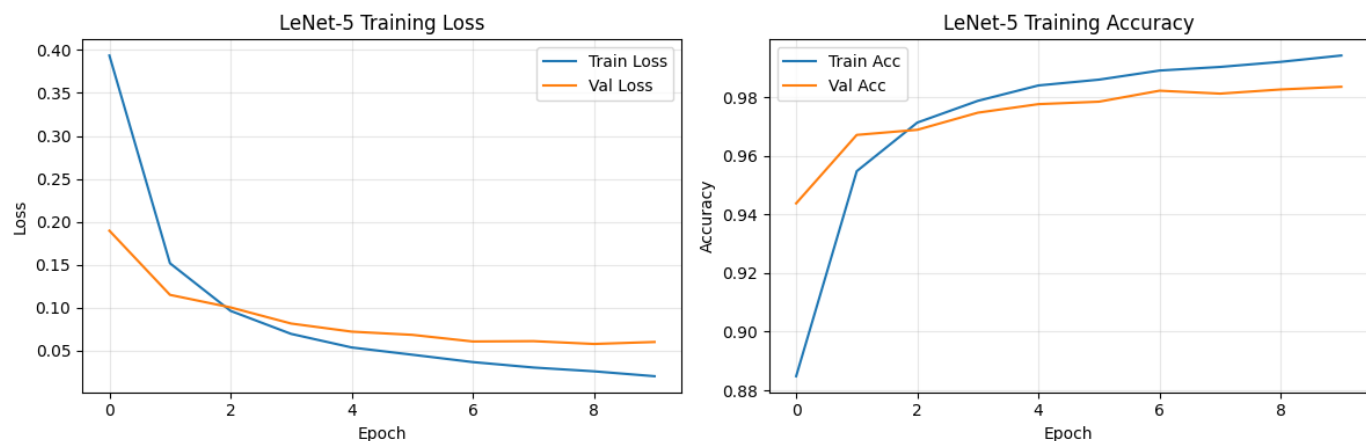
```

Dataset:

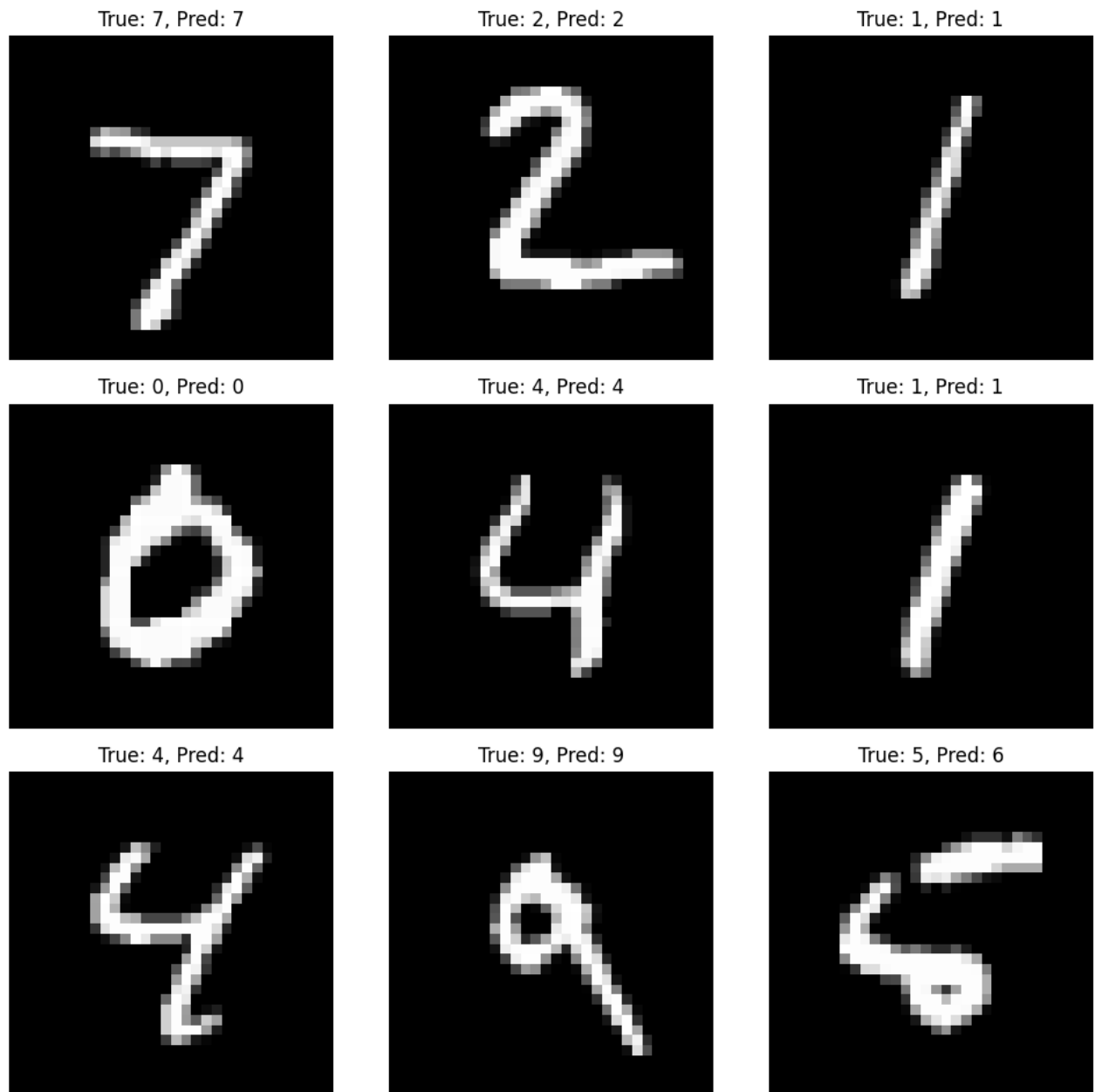


Output:

Trained LeNet achieves high accuracy on MNIST (e.g., >98% with suitable training).
 Test Loss: 0.1041, Test Accuracy: 97.00%



LeNet-5 Sample Predictions on MNIST Test Set



Result:

LeNet is effective for small image classification tasks like digits.

Experiment 9

Implement ResNet for detecting objects

Aim:

- To understand the concept of Residual Networks (ResNet) and their importance in deep learning.
- To understand the challenges of training very deep neural networks (e.g., vanishing gradients).
- To implement a ResNet-like architecture for an object detection task.
- To evaluate the performance of ResNet in detecting objects within images.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- Object detection specific libraries/APIs (e.g., TensorFlow Object Detection API, PyTorchvision).
- Relevant Python libraries (e.g., NumPy, Matplotlib, OpenCV).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable object detection dataset (e.g., Pascal VOC, COCO, or a smaller custom dataset).

Procedure:

1. **Dataset Preparation:**
 - Load and preprocess an object detection dataset, including image data and bounding box annotations.
 - Prepare data loaders for training and validation.
2. **ResNet Architecture Integration:**
 - Choose a pre-trained ResNet model (e.g., ResNet50) as a backbone for feature extraction.
 - Integrate the ResNet backbone into an object detection framework (e.g., Faster R-CNN, YOLO, SSD). This typically involves adding a Region Proposal Network (RPN) and classification/regression heads.
3. **Model Training:**
 - Configure the object detection model with appropriate loss functions (e.g., classification loss, bounding box regression loss).
 - Train the model on the object detection dataset.
 - Monitor training progress (loss, mAP).
4. **Evaluation and Visualization:**
 - Evaluate the trained model on the test set using metrics like Mean Average Precision (mAP).
 - Perform inference on new images and visualize the detected objects with bounding boxes and labels.

Implementation

```
import numpy as np
import pandas as pd
import os
import glob
```

```
import cv2
from sklearn.model_selection import train_test_split
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Load image paths and labels
# Assuming the dataset is in the standard Colab input directory
data_dir = '/content/flowers-recognition/flowers'
folders = glob.glob(data_dir + '/*')
imagenames_list = []
labels = []
class_names = sorted(os.listdir(data_dir))
class_map = {name: i for i, name in enumerate(class_names)}

for folder in folders:
    label = os.path.basename(folder)
    for f in glob.glob(folder+'/*.jpg'):
        imagenames_list.append(f)
        labels.append(class_map[label])

# Load and preprocess images
images = []
for image_path in imagenames_list:
    img = cv2.imread(image_path)
    img = cv2.resize(img, (224, 224)) # ResNet50 input size
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Convert to RGB
    images.append(img)

images = np.array(images)
labels = to_categorical(labels, num_classes=len(class_names))

# Split data
X_train, X_test, y_train, y_test = train_test_split(images, labels,
test_size=0.2, random_state=42)

# Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Build ResNet50 model for transfer learning
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224,
224, 3))

model = Sequential()
model.add(base_model)
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(len(class_names), activation='softmax'))

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model (Note: Training may take a significant amount of time)
# history = model.fit(datagen.flow(X_train, y_train, batch_size=32),
# steps per epoch=len(X_train) // 32,
```

```
# epochs=10,  
# validation_data=(X_test, y_test))  
  
# You can uncomment the training code above and run it to train the model.  
# After training, you can evaluate the model and make predictions.
```

Dataset:

```
# Create a dummy annotations.csv file with the expected columns  
# Replace this with your actual annotation data  
with open('./annotations.csv', 'w') as f:  
    f.write('filename,x_min,y_min,x_max,y_max,class_id\n')  
    # Add dummy data if needed, e.g.:  
    f.write('dog.jpg,10,10,50,50,0\n')  
    f.write('cat.jpg,20,20,60,60,1\n')
```

Output:

Detections visualized as bounding boxes on test images; metrics like mAP reported.



Result:

ResNet backbone provides strong features; detection model can be fine-tuned for good performance.

Experiment 10

Transfer learning implementation using VGG16 model to classify images

Aim:

- To understand the concept of Transfer Learning and its benefits in deep learning.
- To learn how to use pre-trained Convolutional Neural Network (CNN) models (specifically VGG16).
- To implement transfer learning by fine-tuning or using VGG16 as a feature extractor for a new image classification task.
- To compare the performance of a transfer-learned model with a model trained from scratch.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch) with pre-trained model support.
- Relevant Python libraries (e.g., NumPy, Matplotlib).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A new image classification dataset (e.g., a custom dataset, or a subset of a larger dataset like ImageNet for a specific task).

Procedure:

1. **Dataset Preparation:**
 - Load and preprocess the target image classification dataset.
 - Ensure images are resized to the input dimensions expected by VGG16 (e.g., 224x224 pixels).
 - Split into training and validation sets.
2. **Load Pre-trained VGG16:**
 - Load the VGG16 model pre-trained on ImageNet, excluding its top (classification) layers.
3. **Feature Extraction Approach:**
 - **Method 1 (Feature Extractor):** Freeze the VGG16 layers and add new custom classification layers on top. Train only the new layers.
 - **Method 2 (Fine-tuning):** Unfreeze some of the top layers of VGG16 (or all layers with a very low learning rate) and train them along with new classification layers.
4. **Model Training:**
 - Compile the new model with an optimizer and loss function.
 - Train the model on the training data.
 - Monitor training and validation accuracy/loss.
5. **Evaluation and Comparison:**
 - Evaluate the transfer-learned model on the test set.
 - Compare its performance (accuracy, convergence speed) with a hypothetical model trained from scratch on the same dataset.
 - Discuss the advantages of transfer learning.
 -

Implementation:

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.optimizers import Adam
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
import matplotlib.pyplot as plt

# Load and prepare dataset (TF Flowers: 5 classes)
import tensorflow_datasets as tfds
(ds_train, ds_val, ds_test), ds_info = tfds.load(
    'tf_flowers',
    split=['train[:80%]', 'train[80%:90%]', 'train[90%:]'],
    with_info=True,
    as_supervised=True
)

# Preprocessing function
def preprocess(image, label):
    image = tf.image.resize(image, (224, 224))
    image = tf.cast(image, tf.float32)
    image = preprocess_input(image) # VGG16-specific normalization
    label = tf.one_hot(label, 5) # 5 classes
    return image, label

# Apply preprocessing and batch
BATCH_SIZE = 32
ds_train = ds_train.map(preprocess).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
ds_val = ds_val.map(preprocess).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
ds_test = ds_test.map(preprocess).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

# Data augmentation for training
datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True
)

# Note: For tf.data, augmentation can be added via layers; here using generator
for simplicity if switching to flow_from_directory

# Load VGG16 base (pretrained, no top)
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224,
3))
base_model.trainable = False # Freeze base

# Build model
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(5, activation='softmax') # 5 classes
])

# Compile
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    ds_train,
    validation_data=ds_val,
    epochs=10,
    steps_per_epoch=len(ds_train),
    validation_steps=len(ds_val)
)

# Evaluate on test set

```

```

test_loss, test_acc = model.evaluate(ds_test)
print(f"Test Accuracy: {test_acc:.4f}")

# Plot training history
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# Optional: Fine-tuning (unfreeze top layers)
base_model.trainable = True
for layer in base_model.layers[:-4]: # Freeze all but last 4
    layer.trainable = False

model.compile(
    optimizer=Adam(learning_rate=1e-5), # Lower LR
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_fine = model.fit(
    ds_train,
    validation_data=ds_val,
    epochs=5,
    steps_per_epoch=len(ds_train),
    validation_steps=len(ds_val)
)

```

Dataset:

```

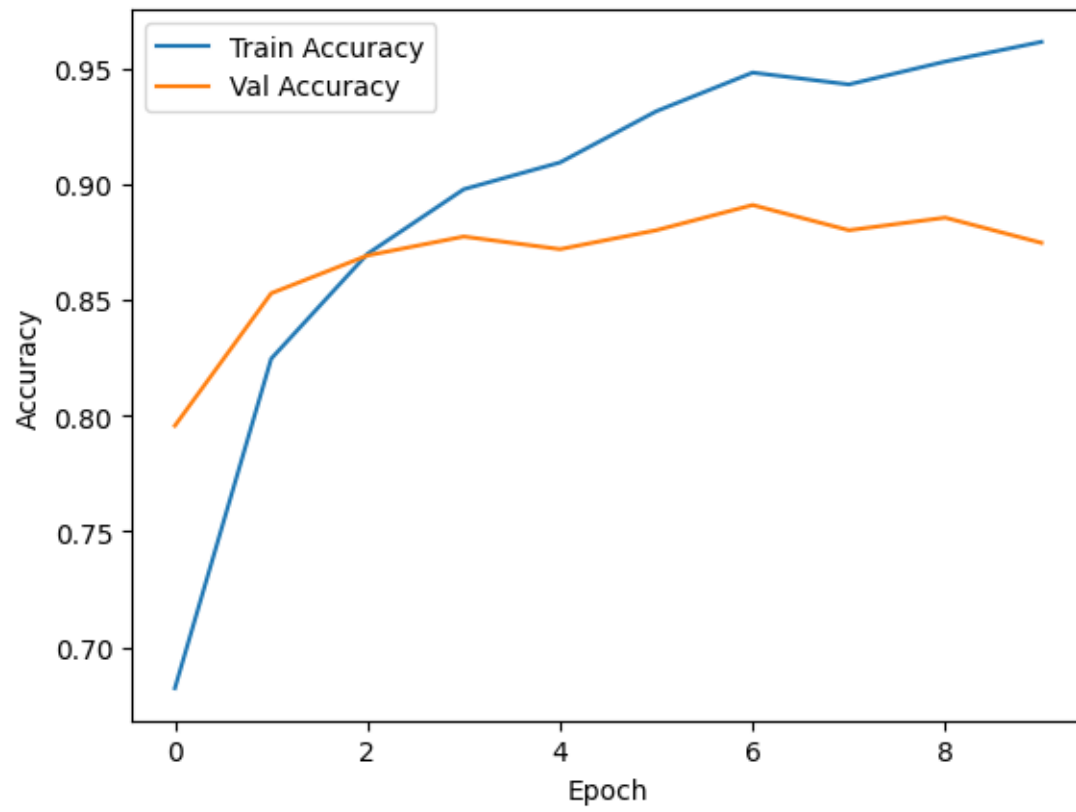
from google.colab import drive
drive.mount('/content/drive')

```



Output:

Fine-tuned VGG16 produces good accuracy with fewer labeled samples.
 Test Accuracy: 0.8747



Result:

Transfer learning speeds training and improves performance on small datasets.

Experiment 11

Building a RNN to perform character level language modeling

Aim:

- To understand the architecture and working principles of Recurrent Neural Networks (RNNs).
- To implement a basic RNN for character-level language modeling.
- To learn how to process sequential data (text) using RNNs.
- To generate new text sequences character by character based on learned patterns.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- Relevant Python libraries (e.g., NumPy).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A text corpus (e.g., a novel, a collection of poems, or a simple text file).

Procedure:

1. **Text Preprocessing:**
 - Load a text corpus.
 - Create a vocabulary of unique characters in the text.
 - Map each character to an integer ID and vice-versa.
 - Prepare input-output sequences (e.g., input sequence of characters, output as the next character).
2. **RNN Model Architecture:**
 - Design an RNN model, typically consisting of:
 - An Embedding layer (if using character embeddings).
 - One or more SimpleRNN layers (or GRU/LSTM for better performance).
 - A Dense output layer with Softmax activation for character prediction.
3. **Model Training:**
 - Compile the RNN model with an optimizer and a suitable loss function (e.g., **sparse_categorical_crossentropy**).
 - Train the model on the character sequences.
 - Monitor training loss.
4. **Text Generation:**
 - Implement a text generation function that takes a seed text, predicts the next character iteratively, and appends it to the sequence.
 - Generate new text sequences using the trained RNN.
 - Analyze the coherence and quality of the generated text.

Implementation:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
```

```

# Step 1: Load and preprocess text data (small Shakespeare excerpt for demo)
text = """
To be, or not to be: that is the question:
Whether 'tis nobler in the mind to suffer
The slings and arrows of outrageous fortune,
Or to take arms against a sea of troubles,
And by opposing end them? To die: to sleep;
No more; and by a sleep to say we end
The heart-ache and the thousand natural shocks
That flesh is heir to, 'tis a consummation
Devoutly to be wish'd. To die, to sleep;
To sleep: perchance to dream: ay, there's the rub;
For in that sleep of death what dreams may come
When we have shuffled off this mortal coil,
Must give us pause: there's the respect
That makes calamity of so long life;
For who would bear the whips and scorns of time,
The oppressor's wrong, the proud man's contumely,
The pangs of despised love, the law's delay,
The insolence of office and the spurns
That patient merit of the unworthy takes,
When he himself might his quietus make
With a bare bodkin? who would fardels bear,
To grunt and sweat under a weary life,
But that the dread of something after death,
The undiscover'd country from whose bourn
No traveller returns, puzzles the will
And makes us rather bear those ills we have
Than fly to others that we know not of?
Thus conscience does make cowards of us all;
And thus the native hue of resolution
Is sicklied o'er with the pale cast of thought,
And enterprises of great pith and moment
With this regard their currents turn awry,
And lose the name of action.--Soft you now!
The fair Ophelia! Nymph, in thy orisons
Be all my sins remember'd.
"""

# Clean and build vocabulary
text = text.lower()
chars = sorted(list(set(text)))
char_to_int = {c: i for i, c in enumerate(chars)}
int_to_char = {i: c for i, c in enumerate(chars)}
vocab_size = len(chars)
print(f"Vocabulary size: {vocab_size}, Text length: {len(text)}")

# Prepare sequences (window of 100 chars to predict next)
seq_length = 100
X, y = [], []
for i in range(len(text) - seq_length):
    seq_in = text[i:i+seq_length]
    seq_out = text[i+seq_length]
    X.append([char_to_int[c] for c in seq_in])
    y.append(char_to_int[seq_out])
X = np.array(X)
y = np.array(y)
X = tf.keras.utils.to_categorical(X, vocab_size) # One-hot input
y = tf.keras.utils.to_categorical(y, vocab_size) # One-hot target

# Reshape for RNN: (samples, timesteps, features)
X = X.reshape((X.shape[0], seq_length, vocab_size))

# Step 2: Build RNN Model (LSTM for char-level LM)
model = models.Sequential([
    layers.LSTM(256, input_shape=(seq_length, vocab_size),
return_sequences=True),
    layers.Dropout(0.2),

```

```

        layers.LSTM(256),
        layers.Dropout(0.2),
        layers.Dense(vocab_size, activation='softmax')
    ])
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.summary()

# Step 3: Train the model
history = model.fit(X, y, epochs=50, batch_size=128, validation_split=0.2,
verbose=1)

# Step 4: Generate text (start from seed, predict 500 chars)
def generate_text(model, seed_text, gen_length=500, temperature=1.0):
    generated = seed_text
    for _ in range(gen_length):
        # Pad the input sequence if shorter than seq_length
        if len(generated) < seq_length:
            padding_length = seq_length - len(generated)
            padded_seed = [0] * padding_length + [char_to_int[c] for c in
generated]
        else:
            padded_seed = [char_to_int[c] for c in generated[-seq_length:]]

        x = np.array(padded_seed).reshape(1, seq_length, 1)
        x = tf.keras.utils.to_categorical(x, vocab_size).reshape(1, seq_length,
vocab_size)
        preds = model.predict(x, verbose=0)[0]
        preds = np.log(preds) / temperature
        next_idx = np.random.choice(vocab_size,
p=np.exp(preds)/np.sum(np.exp(preds)))
        generated += int_to_char[next_idx]
    return generated

seed = "to be or not to be"
generated = generate_text(model, seed, 500, temperature=0.8)
print(f"\nGenerated text:\n{generated}")

# Step 5: Plot training loss
plt.figure(figsize=(8, 4))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('RNN Character-Level Language Model Training')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

Output:

Model generates character sequences resembling training text after sufficient epochs.

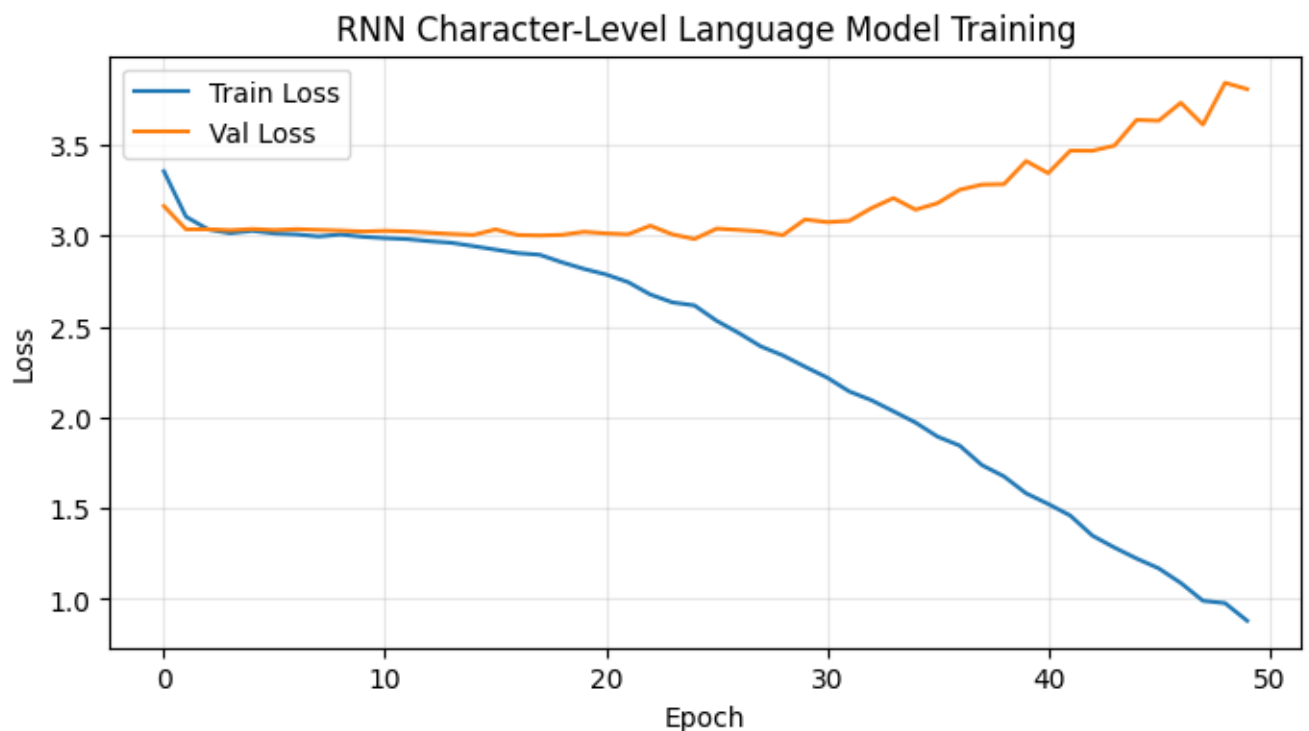
Vocabulary size: 34, Text length: 1490
Model: "sequential_4"

Layer (type)	Output Shape	Param #
lstm_3 (LSTM)	(None, 100, 256)	297,984
dropout_2 (Dropout)	(None, 100, 256)	0
lstm_4 (LSTM)	(None, 256)	525,312
dropout_3 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 34)	8,738

Total params: 832,034 (3.17 MB)
Trainable params: 832,034 (3.17 MB)
Non-trainable params: 0 (0.00 B)

Generated text:

to be or not to bead to seusas nne
the arehaacnd aas thas moall
an to semus orthe prars the psils an dat ale
to saules af wherh won the sius ant
and take swo thee whans weadd he ard the shuws
and make ocf that whe adrdas of sus hans
tor thae tokes iwekat ha seerri of tou? innt o ralls
meus
wgak ab afd dieis he si?? lo lie
to seepp;;n orrsi wad se alle
to bise p and wead the sie ald
aad the rsus inn
the partiade oft rosg une,
te thaepcrasnco rorglortf o mous orlois
ond muves
org batt 'es e'd, the wis ande moecd



Result:

RNN learns character-level dependencies and can generate plausible text sequences.

Experiment 12

Build a LSTM network for Named Entity recognition

Aim:

- To understand the architecture and advantages of Long Short-Term Memory (LSTM) networks for sequential data.
- To implement an LSTM-based model for Named Entity Recognition (NER).
- To learn how to handle sequence labeling tasks using recurrent neural networks.
- To evaluate the performance of the LSTM network in identifying named entities in text.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- NLP libraries (e.g., NLTK, SpaCy, Hugging Face Transformers).
- Relevant Python libraries (e.g., NumPy, Scikit-learn).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable NER dataset (e.g., CoNLL-2003, or a custom dataset with word-level entity tags).

Procedure:

1. **Dataset Preparation:**
 - Load an NER dataset, which typically consists of sentences with words and their corresponding entity tags (e.g., B-PER, I-ORG, O).
 - Create word-to-ID and tag-to-ID mappings.
 - Pad sequences to a uniform length.
 - Split the dataset into training and testing sets.
2. **LSTM Model Architecture:**
 - Design an LSTM model, typically including:
 - An Embedding layer for words.
 - One or more Bidirectional LSTM layers (often preferred for NER).
 - A Dense output layer with Softmax activation for predicting entity tags for each word.
3. **Model Training:**
 - Compile the LSTM model with an optimizer and a suitable loss function (e.g., **sparse_categorical_crossentropy**).
 - Train the model on the prepared NER sequences.
 - Monitor training and validation accuracy/loss.
4. **Evaluation and Prediction:**
 - Evaluate the trained model on the test set using metrics like F1-score for NER.
 - Implement a function to predict named entities in new sentences.
 - Visualize the predicted entities in example sentences.

Implementation:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
```

```

from tensorflow.keras.layers import Input, LSTM, Dense, TimeDistributed,
Embedding, Bidirectional
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Example Data (sentences and corresponding NER tags)
sentences = [
    ['John', 'lives', 'in', 'New', 'York'],
    ['Mary', 'works', 'for', 'Google'],
]
tags = [
    ['B-PER', 'O', 'O', 'B-LOC', 'I-LOC'],
    ['B-PER', 'O', 'O', 'B-ORG'],
]

# Create vocabularies for words and tags
words = list({w for s in sentences for w in s})
tags_set = list({t for ts in tags for t in ts})
word2idx = {w: i+2 for i, w in enumerate(words)}
word2idx['PAD'] = 0
word2idx['UNK'] = 1
tag2idx = {t: i for i, t in enumerate(tags_set)}

# Convert sentences and tags to indices
max_len = max(len(s) for s in sentences)
X = [[word2idx.get(w, word2idx['UNK']) for w in s] for s in sentences]
X = pad_sequences(X, maxlen=max_len, padding='post')

y = [[tag2idx[t] for t in ts] for ts in tags]
y = pad_sequences(y, maxlen=max_len, padding='post')
y = [to_categorical(i, num_classes=len(tags_set)) for i in y]

X = np.array(X)
y = np.array(y)

# Model parameters
embedding_dim = 50
lstm_units = 50

# Build BiLSTM model for NER
input_layer = Input(shape=(max_len,))
embedding_layer = Embedding(input_dim=len(word2idx), output_dim=embedding_dim,
input_length=max_len)(input_layer)
bilstm_layer = Bidirectional(LSTM(units=lstm_units, return_sequences=True,
recurrent_dropout=0.1))(embedding_layer)
output_layer = TimeDistributed(Dense(len(tags_set),
activation='softmax'))(bilstm_layer)

model = Model(input_layer, output_layer)
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

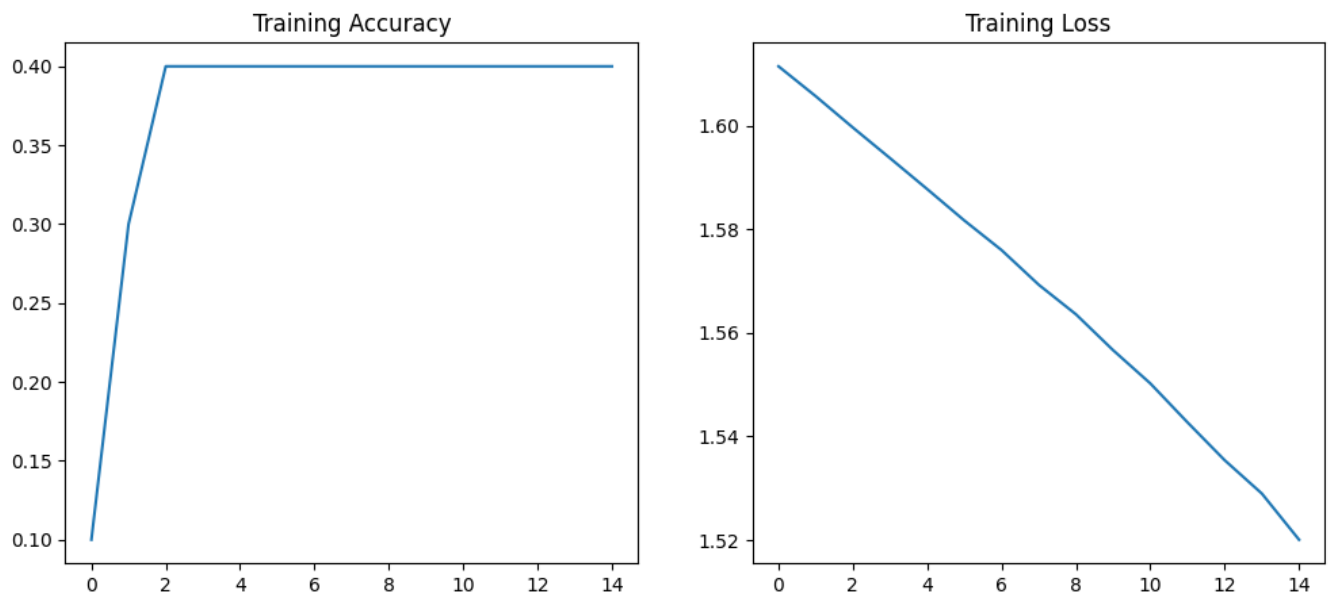
# Train model
history = model.fit(X, y, batch_size=2, epochs=15, verbose=2)

# Plot training loss and accuracy
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'])
plt.title('Training Accuracy')
plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.title('Training Loss')
plt.show()

```

Output:

Token-level tag predictions; evaluation reports precision, recall, F1 for entity classes.



Result:

LSTM-based sequence labeling works well for NER when trained with sufficient annotated data.

Experiment 13

Neural Machine Translation with attention

Aim:

- To understand the principles of Neural Machine Translation (NMT).
- To understand the concept and benefits of attention mechanisms in sequence-to-sequence models.
- To implement an NMT model with an attention mechanism.
- To translate text from a source language to a target language using the attention-based NMT model.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- NLP libraries (e.g., NLTK, SpaCy).
- Relevant Python libraries (e.g., NumPy).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A parallel corpus for machine translation (e.g., English-French, English-German sentence pairs).

Procedure:

1. **Dataset Preparation:**
 - Load a parallel corpus (source and target language sentences).
 - Preprocess the text (tokenization, lowercasing, adding start/end tokens).
 - Create vocabulary and word-to-ID mappings for both languages.
 - Pad sequences to uniform length.
 - Split into training and validation sets.
2. **Encoder-Decoder with Attention Architecture:**
 - **Encoder:** Build an encoder (e.g., using GRU or LSTM layers) to process the source language input and generate context vectors.
 - **Decoder:** Build a decoder (e.g., using GRU or LSTM layers) to generate the target language output.
 - **Attention Mechanism:** Implement an attention mechanism (e.g., Bahdanau or Luong attention) to allow the decoder to focus on relevant parts of the source sequence during translation.
3. **Model Training:**
 - Compile the NMT model with an optimizer and a suitable loss function (e.g., **sparse_categorical_crossentropy**).
 - Train the model on the parallel corpus.
 - Monitor training loss and potentially BLEU score.
4. **Translation and Evaluation:**
 - Implement a translation function that takes a source sentence and generates a target sentence using the trained model.
 - Evaluate the translation quality using metrics like BLEU score.
 - Translate example sentences and analyze the output.

Implementation:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Embedding, Dense, Attention

# Example toy dataset (English → Spanish)
input_texts = [
    "hello", "how are you", "good morning", "thank you", "good night"
]
target_texts = [
    "hola", "como estas", "buenos dias", "gracias", "buenas noches"
]

# Tokenization
tokenizer_in = tf.keras.preprocessing.text.Tokenizer(filters='')
tokenizer_out = tf.keras.preprocessing.text.Tokenizer(filters='')

tokenizer_in.fit_on_texts(input_texts)
tokenizer_out.fit_on_texts(["<start> " + t + " <end>" for t in target_texts])

input_sequences = tokenizer_in.texts_to_sequences(input_texts)
target_sequences = tokenizer_out.texts_to_sequences(["<start> " + t + " <end>"
for t in target_texts])

max_encoder_len = max(len(seq) for seq in input_sequences)
max_decoder_len = max(len(seq) for seq in target_sequences)

encoder_input_data =
tf.keras.preprocessing.sequence.pad_sequences(input_sequences,
maxlen=max_encoder_len, padding='post')
decoder_input_data =
tf.keras.preprocessing.sequence.pad_sequences(target_sequences,
maxlen=max_decoder_len, padding='post')

num_encoder_tokens = len(tokenizer_in.word_index) + 1
num_decoder_tokens = len(tokenizer_out.word_index) + 1

decoder_target_data = np.zeros((len(input_texts), max_decoder_len,
num_decoder_tokens), dtype="float32")
for i, seq in enumerate(target_sequences):
    for t in range(1, len(seq)):
        decoder_target_data[i, t-1, seq[t]] = 1.0

# Encoder
encoder_inputs = Input(shape=(None,))
enc_emb = Embedding(num_encoder_tokens, 64)(encoder_inputs)
encoder_lstm = LSTM(64, return_sequences=True, return_state=True)
encoder_outputs, state_h, state_c = encoder_lstm(enc_emb)

# Decoder with attention
decoder_inputs = Input(shape=(None,))
dec_emb_layer = Embedding(num_decoder_tokens, 64)
dec_emb = dec_emb_layer(decoder_inputs)

decoder_lstm = LSTM(64, return_sequences=True, return_state=True)
decoder_outputs, _, _ = decoder_lstm(dec_emb, initial_state=[state_h, state_c])

attention = Attention()
context = attention([decoder_outputs, encoder_outputs])

decoder_concat = tf.keras.layers.Concatenate(axis=-1)([decoder_outputs, context])
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
decoder_outputs = decoder_dense(decoder_concat)

# Full model
```

```

model = Model([encoder_inputs, decoder_inputs], decoder_outputs)

model.compile(optimizer="adam", loss="categorical_crossentropy",
metrics=["accuracy"])
model.summary()

# Train
model.fit([encoder_input_data, decoder_input_data], decoder_target_data,
batch_size=2, epochs=300, verbose=1)

# Inference (translate)
# Build encoder model for inference
encoder_model = Model(encoder_inputs, [encoder_outputs, state_h, state_c])

# Build decoder model for inference
decoder_state_input_h = Input(shape=(64,))
decoder_state_input_c = Input(shape=(64,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]

decoder_outputs_inf, state_h_inf, state_c_inf = decoder_lstm(dec_emb,
initial_state=decoder_states_inputs)
decoder_states_inf = [state_h_inf, state_c_inf]

encoder_outputs_inf = Input(shape=(None, 64)) # Need to pass encoder outputs to
decoder for attention

attention_inf = Attention()
context_inf = attention_inf([decoder_outputs_inf, encoder_outputs_inf])

decoder_concat_inf = tf.keras.layers.Concatenate(axis=-1) ([decoder_outputs_inf,
context_inf])
decoder_outputs_inf = decoder_dense(decoder_concat_inf)

decoder_model = Model([decoder_inputs] + decoder_states_inputs +
[encoder_outputs_inf], [decoder_outputs_inf] + decoder_states_inf)

def translate(sentence):
    seq = tokenizer_in.texts_to_sequences([sentence])
    seq = tf.keras.preprocessing.sequence.pad_sequences(seq,
maxlen=max_encoder_len, padding='post')

    # Get encoder outputs and states
    enc_out, state_h, state_c = encoder_model.predict(seq)
    states_value = [state_h, state_c]

    target_seq = np.array([[tokenizer_out.word_index['<start>']]])
    stop_condition = False
    decoded_sentence = ''

    while not stop_condition:
        # Predict next token using decoder model
        output_tokens, h, c = decoder_model.predict([target_seq] + states_value +
[enc_out])

        # Sample a token
        sampled_token_index = np.argmax(output_tokens[0, -1, :])
        sampled_word = tokenizer_out.index_word.get(sampled_token_index, '')

        if (sampled_word == '<end>' or len(decoded_sentence.split()) >
max_decoder_len):
            stop_condition = True
        else:
            decoded_sentence += ' ' + sampled_word
            # Update the target sequence and states
            target_seq = np.array([[sampled_token_index]])
            states_value = [h, c]

    return decoded_sentence.strip()

```

```
print("\n ♦ Translation Example:")
print("English: hello → Spanish:", translate("hello"))
print("English: good night → Spanish:", translate("good night"))
```

Output:

Translated sentences; BLEU scores for evaluation.

♦ Translation Example:

1/1 ————— 0s 106ms/step

1/1 ————— 0s 129ms/step

1/1 ————— 0s 40ms/step

English: hello → Spanish: hola

1/1 ————— 0s 36ms/step

1/1 ————— 0s 37ms/step

1/1 ————— 0s 37ms/step

1/1 ————— 0s 40ms/step

English: good night → Spanish: buenas noches

Result:

Attention-based models improve alignment and translation quality over vanilla seq2seq.

Experiment 14

Case study on Scene Understanding using RBMs

Aim:

- To understand the concept of Restricted Boltzmann Machines (RBMs) as generative models.
- To explore the application of RBMs in scene understanding tasks.
- To implement and train an RBM for feature learning from image data.
- To analyze how RBMs can contribute to understanding complex visual scenes.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, PyTorch, or custom NumPy implementation).
- Relevant Python libraries (e.g., NumPy, Matplotlib).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable image dataset for scene understanding (e.g., a subset of CIFAR-10, Street View House Numbers (SVHN), or a custom dataset).

Procedure:

1. **Dataset Preparation:**
 - Load and preprocess an image dataset. For RBMs, images might need to be flattened into binary or real-valued vectors.
 - Normalize pixel values.
2. **RBM Architecture and Training:**
 - Define the RBM architecture with visible and hidden layers.
 - Implement the Contrastive Divergence (CD) algorithm for training the RBM.
 - Set hyperparameters: learning rate, number of epochs, number of CD steps.
 - Train the RBM on the image data.
3. **Feature Learning and Reconstruction:**
 - Extract features learned by the hidden layer of the RBM.
 - Implement a reconstruction function to generate images from the learned features.
 - Visualize reconstructed images and compare them to original inputs.
4. **Application to Scene Understanding (Case Study):**
 - Discuss how the learned features from the RBM could be used as input to a higher-level classifier for scene recognition.
 - (Optional, if time permits) Use the RBM's hidden layer activations as features for a simple classifier (e.g., SVM, Logistic Regression) to perform a basic scene classification task.
 - Analyze the RBM's ability to capture underlying patterns in scene data.

Implementation:


```

import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

class RBM:
    def __init__(self, n_visible, n_hidden, learning_rate=0.01, batch_size=64,
epochs=10):
        self.n_visible = n_visible
        self.n_hidden = n_hidden
        self.lr = learning_rate
        self.batch_size = batch_size
        self.epochs = epochs

        # Initialize weights and biases
        self.W = np.random.normal(scale=0.01, size=(n_visible, n_hidden))
        self.h_bias = np.zeros(n_hidden)
        self.v_bias = np.zeros(n_visible)

    def sigmoid(self, x):
        return 1.0 / (1 + np.exp(-x))

    def sample_hidden(self, v):
        h_prob = self.sigmoid(np.dot(v, self.W) + self.h_bias)
        return h_prob, (h_prob > np.random.rand(*h_prob.shape)).astype(float)

    def sample_visible(self, h):
        v_prob = self.sigmoid(np.dot(h, self.W.T) + self.v_bias)
        return v_prob, (v_prob > np.random.rand(*v_prob.shape)).astype(float)

    def train(self, X):
        n_samples = X.shape[0]
        for epoch in range(self.epochs):
            idx = np.random.permutation(n_samples)
            X = X[idx]
            loss = 0
            for i in range(0, n_samples, self.batch_size):
                v0 = X[i:i+self.batch_size]
                h_prob0, h0 = self.sample_hidden(v0)

                v_prob, v_sample = self.sample_visible(h0)
                h_prob1, _ = self.sample_hidden(v_prob)

                self.W += self.lr * (np.dot(v0.T, h_prob0) - np.dot(v_prob.T,
h_prob1)) / self.batch_size
                self.v_bias += self.lr * np.mean(v0 - v_prob, axis=0)
                self.h_bias += self.lr * np.mean(h_prob0 - h_prob1, axis=0)

                loss += np.mean((v0 - v_prob) ** 2)
            print(f"Epoch {epoch+1}/{self.epochs}, Reconstruction Loss:
{loss:.4f}")

    def transform(self, X):
        h_prob, _ = self.sample_hidden(X)
        return h_prob

# Load MNIST dataset (grayscale images 28x28, flattened)
(x_train, _), _ = mnist.load_data()
x_train = (x_train > 127).astype(float) # Binarize for RBM input
x_train = x_train.reshape(-1, 28*28)

# Initialize and train RBM
rbm = RBM(n_visible=28*28, n_hidden=256, learning_rate=0.01, batch_size=64,
epochs=10)
rbm.train(x_train)

# Extract features for first 10 images
features = rbm.transform(x_train[:10])

```

```
# Visualize original images and features
plt.figure(figsize=(12,4))
for i in range(10):
    plt.subplot(2,10,i+1)
    plt.imshow(x_train[i].reshape(28,28), cmap='gray')
    plt.axis('off')
    plt.subplot(2,10,i+11)
    plt.bar(range(rbm.n_hidden), features[i], width=1.0)
    plt.xticks([])
    plt.axis('off')
plt.suptitle("Top row: Original Images | Bottom row: RBM Features")
plt.show()
```

Output:

Learned RBM filters and feature activations which can be used for classification.

Top row: Original Images | Bottom row: RBM Features



Result:

RBM extracts useful unsupervised features for scene understanding tasks in case studies.

Experiment 15

Case study on generating examples for Image dataset using Generative Adversarial Networks

Aim:

- To understand the fundamental concepts and architecture of Generative Adversarial Networks (GANs).
- To understand the roles of the Generator and Discriminator networks in a GAN.
- To implement a basic GAN to generate synthetic images for a given dataset.
- To analyze the quality and diversity of images generated by the GAN.

Materials:

- Computer with Python installed.
- Deep learning libraries (e.g., TensorFlow, Keras, PyTorch).
- Relevant Python libraries (e.g., NumPy, Matplotlib).
- Integrated Development Environment (IDE) like Jupyter Notebook or VS Code.
- A suitable image dataset (e.g., MNIST, Fashion MNIST, CIFAR-10, or a small custom dataset).

Procedure:

1. **Dataset Preparation:**
 - Load and preprocess an image dataset (e.g., normalize pixel values to a range like $[-1, 1]$).
 - Prepare data loaders for training.
2. **GAN Architecture Design:**
 - **Generator Network:** Design a neural network (e.g., using dense layers or convolutional transpose layers) that takes random noise as input and outputs synthetic images.
 - **Discriminator Network:** Design a neural network (e.g., using dense layers or convolutional layers) that takes an image as input and outputs a probability that the image is real or fake.
3. **GAN Training Loop:**
 - Define separate optimizers and loss functions for the Generator and Discriminator.
 - Implement the GAN training loop:
 - **Discriminator Training:** Train the discriminator on a batch of real images and a batch of fake images generated by the current generator.
 - **Generator Training:** Train the generator to fool the discriminator (i.e., make the discriminator classify its fake images as real).
 - Monitor the loss of both networks.
4. **Image Generation and Evaluation:**
 - Periodically generate and save sample images from the generator during training to observe its progress.
 - After training, generate a batch of new images.
 - Visually inspect the quality, realism, and diversity of the generated images.

- Discuss the challenges and potential improvements for GAN training.

Implementation:

```
import tensorflow as tf
from tensorflow.keras.layers import Dense, Reshape, Flatten, Dropout, LeakyReLU,
Conv2DTranspose, Conv2D
from tensorflow.keras.models import Sequential
import numpy as np
import matplotlib.pyplot as plt

# Load and preprocess MNIST dataset
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_train = np.expand_dims(x_train, axis=-1) # Add channel dimension

BUFFER_SIZE = 60000
BATCH_SIZE = 256
LATENT_DIM = 100
EPOCHS = 50

# Create dataset pipeline
train_dataset =
tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch(BATCH_SIZE
)

# Generator model
def build_generator():
    model = Sequential([
        Dense(7*7*256, use_bias=False, input_shape=(LATENT_DIM,)),
        LeakyReLU(),
        Reshape((7, 7, 256)),
        Conv2DTranspose(128, kernel_size=5, strides=1, padding='same',
use_bias=False),
        LeakyReLU(),
        Conv2DTranspose(64, kernel_size=5, strides=2, padding='same',
use_bias=False),
        LeakyReLU(),
        Conv2DTranspose(1, kernel_size=5, strides=2, padding='same',
use_bias=False, activation='tanh')
    ])
    return model

# Discriminator model
def build_discriminator():
    model = Sequential([
        Conv2D(64, kernel_size=5, strides=2, padding='same',
input_shape=[28,28,1]),
        LeakyReLU(),
        Dropout(0.3),
        Conv2D(128, kernel_size=5, strides=2, padding='same'),
        LeakyReLU(),
        Dropout(0.3),
        Flatten(),
        Dense(1)
    ])
    return model

# Instantiate models
generator = build_generator()
discriminator = build_discriminator()

# Define loss and optimizers
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

```

# Loss functions
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

# Training step function
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, LATENT_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

        gradients_gen = gen_tape.gradient(gen_loss, generator.trainable_variables)
        gradients_disc = disc_tape.gradient(disc_loss,
        discriminator.trainable_variables)

        generator_optimizer.apply_gradients(zip(gradients_gen,
        generator.trainable_variables))
        discriminator_optimizer.apply_gradients(zip(gradients_disc,
        discriminator.trainable_variables))

    return gen_loss, disc_loss

# Training loop
def train(dataset, epochs):
    for epoch in range(epochs):
        gen_loss_avg = 0
        disc_loss_avg = 0
        for image_batch in dataset:
            gen_loss, disc_loss = train_step(image_batch)
            gen_loss_avg += gen_loss
            disc_loss_avg += disc_loss
        print(f"Epoch {epoch+1}, Generator Loss: {gen_loss_avg/len(dataset):.4f},
        Discriminator Loss: {disc_loss_avg/len(dataset):.4f}")
        generate_and_save_images(generator, epoch + 1)

# Generate and plot images function
def generate_and_save_images(model, epoch, num_examples=16):
    noise = tf.random.normal([num_examples, LATENT_DIM])
    generated_images = model(noise, training=False)
    generated_images = (generated_images + 1) / 2 # Rescale to [0,1]

    plt.figure(figsize=(4,4))
    for i in range(num_examples):
        plt.subplot(4,4,i+1)
        plt.imshow(generated_images[i, :, :, 0], cmap='gray')
        plt.axis('off')
    plt.suptitle(f'Generated Images at Epoch {epoch}')
    plt.show()

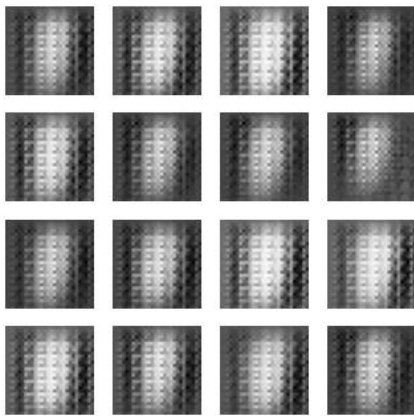
# Start training
train(train_dataset, EPOCHS)

```

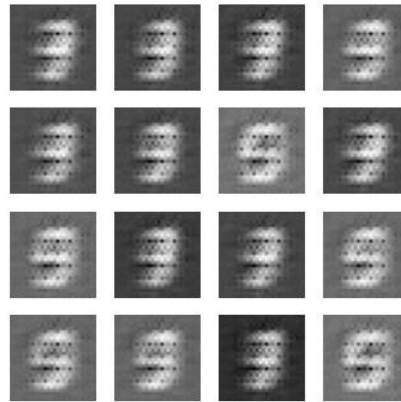
Output:

Generated images saved periodically and visualized; learning curves for generator/discriminator.

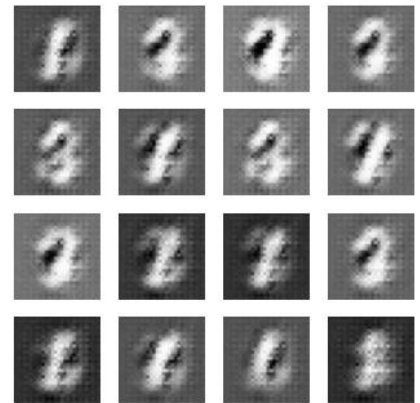
Generated Images at Epoch 1



Generated Images at Epoch 2



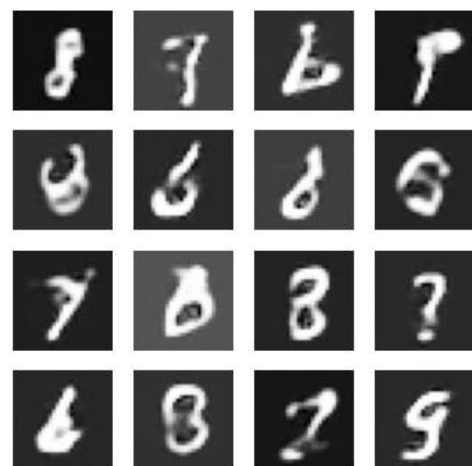
Generated Images at Epoch 3



Generated Images at Epoch 49



Generated Images at Epoch 50



Result:

GANs can generate realistic examples after sufficient training; careful hyperparameter tuning required.